

Escola Naval / Curso EN-AEL

Projeto de Algoritmos e Estruturas de Dados

Grafo de Conhecimento para *Cyber Threat Intelligence*

O projeto consiste no desenvolvimento incremental de uma aplicação que representa conhecimento sobre ameaças cibernéticas através de um grafo. A solução deverá permitir modelar entidades como atores, *malware*, indicadores, campanhas e vulnerabilidades, bem como as relações entre elas. O trabalho começa com uma implementação em memória e evolui progressivamente para persistência, algoritmos de análise, acesso e interface gráfica demonstrativa.

O resultado final esperado é o de uma aplicação funcional, simplificada e demonstrável, capaz de mostrar como um grafo de conhecimento pode ajudar a organizar e interpretar informação dispersa sobre ameaças cibernéticas.

1. Motivação

Todos os dias, organizações públicas e privadas enfrentam sinais dispersos de atividade maliciosa: um endereço IP suspeito, um ficheiro com determinado hash, uma vulnerabilidade explorada, uma campanha de *phishing* ou uma referência a um grupo de atacantes. Isoladamente, cada elemento diz pouco. O verdadeiro valor surge quando conseguimos ligar essas peças e perceber a história que contam em conjunto.

Este projeto propõe a construção de uma versão académica e simplificada de um grafo de conhecimento para inteligência de ameaças cibernéticas. A ideia central é representar conhecimento sobre ameaças como uma rede de entidades relacionadas: atores maliciosos, *malware*, indicadores de compromisso, vulnerabilidades, campanhas, organizações-alvo e técnicas de ataque. Em vez de guardar os dados como uma simples lista, o sistema deve permitir responder a perguntas como: “Que *malware* está associado a este indicador?”, “Que ator usa esta técnica?”, “Existe um caminho entre este IP suspeito e uma campanha conhecida?” ou “Quais são as entidades mais influentes no grafo?”.

O projeto define o objetivo de compreender estruturas de dados, algoritmos e decisões de desenho com detalhe suficiente para produzir uma reimplementação funcional, ainda que simplificada, do subsistema de *knowledge graph*.

A componente técnica é progressiva. O ponto de partida é simples: criar nós, relações e consultas básicas. Depois, o projeto evolui para persistência, algoritmos de análise, acesso local/API e, no nível mais avançado, uma interface gráfica demonstrativa que permita visualizar e explorar a solução.

2. Objetivo geral

Desenvolver uma aplicação acadêmica que implemente um grafo de conhecimento simplificado para *Cyber Threat Intelligence*, permitindo criar, guardar, consultar, analisar e visualizar relações entre entidades de segurança.

3. Objetivos específicos

No final do projeto, o aluno deverá ser capaz de:

1. Representar conhecimento de *threat intelligence* através de nós e relações.
2. Distinguir diferentes tipos de entidades, como atores, malware, indicadores, vulnerabilidades, campanhas e organizações.
3. Validar relações entre entidades segundo uma matriz de relações permitidas.
4. Implementar uma estrutura de grafo em memória usando listas de adjacência.
5. Criar dados sintéticos para testar o comportamento do sistema.
6. Implementar consultas básicas ao grafo, como vizinhos de um nó e caminhos entre entidades.
7. Acrescentar persistência em base de dados.
8. Implementar algoritmos de análise, como caminho mais curto, resolução de entidades, *PageRank* ou detecção de comunidades.
9. Expor funcionalidades através de acesso local/API.
10. Criar uma interface gráfica simples para demonstrar visualmente a solução.

4. Plano de realização do trabalho

O projeto deve ser desenvolvido de forma incremental. Cada nível acrescenta novas funcionalidades ao nível anterior. Assim, para obter uma classificação mais elevada, o aluno deve cumprir também os requisitos dos níveis anteriores.

Nível 1 — Classificação máxima: 12 valores

Tema: Grafo básico em memória

Este é o nível mínimo esperado. O objetivo é demonstrar que o aluno compreendeu a estrutura fundamental do problema: entidades, relações e navegação básica no grafo.

Sub-objetivos:

- Criar uma estrutura de dados para representar nós do grafo.
- Criar uma estrutura de dados para representar relações dirigidas entre nós.

- Implementar uma classe principal para gerir o grafo.
- Permitir adicionar nós e relações.
- Implementar validação simples de tipos de entidades.
- Implementar uma matriz básica de relações válidas, por exemplo:


```

      threat-actor uses malware
      indicator indicates malware
      campaign targets identity
      malware uses attack-pattern
      
```
- Criar um conjunto inicial de dados sintéticos.
- Implementar uma função para listar vizinhos de um nó.
- Implementar testes básicos que demonstrem que os nós, relações e consultas funcionam.

Entregáveis esperados:

- Código-fonte.
- *Dataset* sintético.
- Pequeno relatório explicativo.
- Demonstração por consola.

Nível 2 — Classificação máxima: 14 valores

Tema: Persistência dos dados

Neste nível, o sistema deixa de existir apenas em memória. O aluno deve garantir que o grafo pode ser guardado e recuperado entre execuções.

Sub-objetivos adicionais:

- Criar uma base de dados SQLite .
- Definir tabelas para nós e relações.
- Guardar objetos do grafo na base de dados.
- Carregar o grafo a partir da base de dados.
- Implementar pesquisa de nós por tipo.
- Implementar pesquisa de relações por origem, destino ou tipo.
- Garantir que o grafo carregado da base de dados preserva a estrutura original.
- Acrescentar testes de persistência.

Entregáveis esperados:

- Script de criação da base de dados.

- Código de gravação e carregamento.
- Testes de persistência.
- Relatório atualizado com descrição do modelo de dados.

Nível 3 — Classificação máxima: 16 valores

Tema: Algoritmos de análise do grafo

Neste nível, o projeto passa de uma estrutura de armazenamento para uma ferramenta analítica. O sistema deve permitir extrair conhecimento a partir das ligações existentes.

Sub-objetivos adicionais:

- Implementar procura do caminho mais curto entre duas entidades.
- Implementar pesquisa em largura, por exemplo BFS.
- Implementar pesquisa em profundidade, por exemplo DFS.
- Implementar um algoritmo simples de *PageRank* para identificar entidades centrais.
- Implementar resolução básica de entidades, por exemplo detetar que “APT29” e “Cozy Bear” podem representar o mesmo ator.
- Implementar pelo menos uma consulta analítica, como:
 - “Que atores usam determinado malware?”
 - “Que indicadores apontam para determinada ameaça?”
 - “Qual o caminho entre um indicador e um ator?”
 - “Quais são as entidades mais relevantes no grafo?”
- Apresentar resultados dos algoritmos com base nos dados sintéticos.

Entregáveis esperados:

- Código dos algoritmos.
- Testes de cada algoritmo.
- Exemplos de execução.
- Relatório com interpretação dos resultados obtidos.

Nível 4 — Classificação máxima: 18 valores

Tema: Acesso local ou por API e arquitetura modular

Neste nível, o sistema deve ser organizado como uma aplicação reutilizável, com separação entre lógica do grafo, persistência e interface de acesso.

Sub-objetivos adicionais:

- Organizar o código em módulos ou camadas.
- Criar um acesso local direto ou uma API REST usando FastAPI ou tecnologia equivalente.
- Implementar endpoints mínimos:
 - criar nó;
 - obter nó por identificador;
 - listar nós por tipo;
 - criar relação;
 - listar vizinhos de um nó;
 - calcular caminho entre dois nós;
 - obter ranking de entidades;
 - obter estatísticas gerais do grafo.
- Validar dados de entrada.
- Tratar erros de forma clara, por exemplo entidade inexistente ou relação inválida.
- Criar documentação simples da API.
- Opcionalmente, implementar uma fila de eventos simples para registrar alterações no grafo.

Entregáveis esperados:

- Acesso local ou API funcional.
- Documentação dos endpoints.
- Exemplos de chamadas à API.
- Testes de integração.
- Relatório com descrição da arquitetura.

Nível 5 — Classificação máxima: 20 valores

Tema: Interface gráfica demonstrativa da solução

Este é o nível mais completo. O objetivo é construir uma demonstração visual que permita compreender o valor do grafo sem depender apenas de consola ou chamadas locais/API.

Sub-objetivos adicionais:

- Criar uma interface gráfica simples, web ou desktop.

- Permitir visualizar nós e relações do grafo.
- Permitir filtrar entidades por tipo, por exemplo atores, malware, indicadores ou campanhas.
- Permitir selecionar um nó e ver os seus detalhes.
- Permitir visualizar os vizinhos de um nó.
- Permitir calcular e mostrar o caminho entre duas entidades.
- Apresentar resultados de *PageRank* ou centralidade (identificar entidades mais influentes no grafo. Ver os links [1]) [2].
- Representar visualmente diferentes tipos de nós e relações.
- Integrar a interface com a acesso local/API desenvolvida no nível anterior.
- Preparar uma demonstração final com um cenário narrativo, por exemplo:
 - um indicador suspeito é introduzido;
 - o sistema mostra o malware associado;
 - o grafo revela o ator provável;
 - a interface mostra o caminho de relações que justifica essa conclusão.

Entregáveis esperados:

- Interface gráfica funcional.
- Integração com a acesso local/API.
- Demonstração visual do grafo.
- Relatório final.
- Demonstração do sistema.

5. Cronograma sugerido

Semana	Trabalho previsto	Resultado esperado
1	Definição dos tipos de entidades e relações	Modelo conceptual
1	Implementação do grafo em memória	Nível 1
2	Dados sintéticos, testes e persistência	Nível 2
2	Algoritmos de análise	Nível 3
3	Acesso local ou API e organização modular	Nível 4
4	Interface gráfica, integração	Nível 5

6. Critérios transversais de avaliação

Além das funcionalidades por nível, a avaliação considera:

- Clareza da arquitetura.

- Organização do código.
- Correção das estruturas de dados.
- Validação das relações entre entidades.
- Qualidade dos testes.
- Qualidade dos dados sintéticos.
- Capacidade de explicar as decisões tomadas.
- Relatório técnico claro, mas não extenso.
- Demonstração funcional.
- Capacidade de relacionar a implementação com o enunciado fornecido.